A Lock-Free Ordered List with BST Hints and Efficient Extrema Operations

Combining a Hinted Doubly-Linked List with Unbalanced and Balanced BST Hints

Research Project Report

Roshan Yogendra
Cybersecurity Master, EPFL
roshan@epfl.ch

Supervisor: Gal Sela, Scientist, Distributed Computing Lab, EPFL

Principal Investigator: Prof. Rachid Guerraoui, Distributed Computing Laboratory, EPFL

May 2026

Abstract

This report presents a lock-free ordered list that supports `insert`, `remove`, `min`, `max`, `extract_min`, and `extract_max`. The main idea is to keep correctness in the list and use a binary search tree only as a hint index. The list stores the keys in sorted order. The tree stores pointers into the list so that operations can start close to their target key.

The implementation builds on the Ordered Hinted List of Melnikov and Sela. This project adds destructive extrema operations and compares two list backends with two hint indexes. The barrier-based backend repairs backward pointers with compare-and-swap operations over wrapper objects. The store-based backend repairs backward pointers with a single atomic store of a raw node pointer. The hint index is either an unbalanced lock-free BST or a balanced chromatic tree adapted from the CBAT and SetBench codebase.

All four variants were benchmarked on a 96-core EPFL server. The store-based unbalanced variant is the strongest implementation for destructive extrema operations. The balanced hint index does not improve throughput in the measured workloads because its synchronization and rebalancing cost is larger than the traversal time it saves.

Contents

1 Introduction **3**

2 Background and Related Work **3**
- 2.1 Lock-free ordered lists . . . 3
- 2.2 The Ordered Hinted List of Melnikov and Sela . . . 4
- 2.3 Pragmatic backward pointers . . . 4
- 2.4 BST hint indexes . . . 4

3 Design Overview **5**
- 3.1 List-Based Correctness with Tree-Based Hints . . . 5
- 3.2 Node representation . . . 5
- 3.3 The four variants . . . 6

4 Operations **6**
- 4.1 Search, insert, and remove . . . 6
- 4.2 Minimum and maximum queries . . . 7
- 4.3 Extracting the minimum . . . 8
- 4.4 Extracting the maximum . . . 8

5 Correctness Argument **9**
- 5.1 Linearization points . . . 9
- 5.2 Lock-freedom . . . 9

6 Implementation Notes **10**
- 6.1 Backward-pointer maintenance . . . 10
- 6.2 Balanced BST hint implementation . . . 10
- 6.3 Memory reclamation . . . 11

7 Evaluation **11**
- 7.1 Experimental setup . . . 11
- 7.2 Baseline selection . . . 12
- 7.3 Mixed workload . . . 12
- 7.4 Comparison with a lock-free balanced tree . . . 13
- 7.5 Per-operation results . . . 14

8 Discussion **15**

9 Limitations and Future Work **15**

10 Conclusion **16**

1 Introduction

Many concurrent applications need an ordered structure rather than a hash table. Examples include schedulers, graph algorithms and systems that repeatedly take the smallest or largest available key. These applications need update operations such as `insert` and `remove`. They also need fast access to the ends of the order through `min`, `max`, `extract_min`, and `extract_max`.

A lock-free linked list is a natural starting point. It is ordered and can be made lock-free. The drawback is traversal cost. If the list is sorted in increasing order, the minimum is near the head, but the maximum is near the tail. Without extra information, `max` and `extract_max` require a full forward scan.

A balanced search tree improves search time, but a fully concurrent balanced tree is more complex. Augmented trees can also support queries such as minimum, maximum, rank, and range queries by storing extra information in internal nodes. However, updates must maintain this information while other threads may be updating and rotating the tree. Another issue is the memory required to keep all the extra information from an augmentation.

This project studies a simpler structure. The list is the source of truth. A key is present exactly when a live node for that key appears on the forward list from `head` to `tail`. The BST is only a hint. It returns a nearby list node so that a list operation can start close to its target. A stale or inaccurate hint is allowed.

Contributions. This report makes the following contributions.

1. It implements a lock-free ordered list with a BST hint index.
2. It supports both non-destructive extrema queries and destructive extrema operations.
3. It compares barrier-based and store-based maintenance of backward list pointers.
4. It evaluates two hint indexes: an unbalanced BST and a balanced chromatic tree adapted from the CBAT codebase.
5. It reports benchmark measurements for the four variants.
6. It compares the implementation with CBAT, a lock-free concurrent balanced augmented tree.

2 Background and Related Work

2.1 Lock-free ordered lists

Harris's lock-free list is a standard building block for ordered concurrent lists [4]. Each node has a forward pointer. Deletion has two steps. First, the node is logically deleted by marking its forward pointer. Then the node is physically removed by changing the predecessor's forward pointer. This split lets other threads help finish removals.

The problem for this project is that a singly-linked list is asymmetric. Traversing from `head` is easy, but finding the last live node before `tail` is expensive. A doubly-linked list can help, but exact backward pointers are hard to maintain in a lock-free structure. Fomitchev and Ruppert studied lock-free linked lists and skip lists with additional links and helping mechanisms [3]. This project uses a more specialized approach: backward pointers are hints, while the forward chain remains the source of truth.

2.2 The Ordered Hinted List of Melnikov and Sela

The main starting point is the work of Melnikov and Sela on concurrent extrema and predecessor/successor queries in search trees [5]. Their key idea is to connect the elements of a search tree into a separate ordered list. The tree is still useful because it can return an element close to the target key. The list then implements successor, predecessor, minimum, and maximum operations.

They call this structure an Ordered Hinted List. Like Harris's list, it uses a forward `next` pointer to control membership and deletion. The pointer stores both a node pointer and a state bit such as `Ok` or `Deleted`. Each node also has a backward `prev` pointer. This backward pointer does not need to be exact at every moment. It only needs to remain useful enough that an operation can move backward to a reasonable starting point and then move forward on the real list to validate the result.

This separation is important. The tree does not need to return the true predecessor or successor. It only needs to return a nearby node. Melnikov and Sela use this property to combine the list with different search trees. Even a failed tree search can be useful if it returns a node near the target key. The tree chooses where traversal starts. The list decides what the operation returns.

Their list uses barriers to repair backward pointers. A barrier on a node checks the backward pointer of the node's successor and tries to repair it if it skips past the current node. Barriers are used before insertions and before physical deletion. Their proof shows that active nodes keep backward links that are useful enough for traversal, even if the links are temporarily behind concurrent updates.

This project follows the same overall integration with a tree. To insert a key, the algorithm first asks the tree for a close node, inserts the new node into the hinted list using that node, and then inserts a pointer to the list node into the tree. To delete a key, it first removes the key from the list and then removes the tree entry. Minimum and maximum are implemented from the two sentinels. The main additions are destructive extrema operations, a store-based backward-pointer variant, and an evaluation of both unbalanced and balanced BST hints.

2.3 Pragmatic backward pointers

Träff and Pöter observed that backward pointers in a lock-free doubly-linked list can be helper information rather than correctness-critical state [7]. If correctness does not depend on an exact backward pointer, a repair does not always need a compare-and-swap loop. A plain atomic write can be enough.

The store-based variant applies this idea to the Ordered Hinted List. The forward `next` pointer still controls membership, ordering, and linearization. The backward `prev` pointer only helps choose a starting point for traversal. A stale backward pointer can increase traversal time, but it cannot change the abstract result of an operation.

2.4 BST hint indexes

This project evaluates two choices for the BST hint index. The first is based on the lock-free external BST of Natarajan and Mittal [6]. This tree has low update overhead, but it is not balanced. Since the tree is only used as a hint index, imbalance does not affect correctness. It only affects how close the returned hint is to the target key.

The second choice is a balanced chromatic tree. A chromatic tree is a relaxed red-black tree, and the implementation used here is adapted from SetBench [8] and the CBAT codebase [9]. It follows the lock-free tree technique of Brown, Ellen, and Ruppert [2]. Their method uses LLX and SCX operations to replace small parts of a tree atomically. At a high level, LLX reads a stable snapshot of a node, while SCX uses those snapshots to update one pointer and finalize nodes that are removed.

In this project, the balanced tree is not used as a generic augmented tree. It is only used as a balanced map from keys to list-node pointers. This keeps the design simpler than CBAT, where queries rely on correctly maintained augmented values.

3 Design Overview

3.1 List-Based Correctness with Tree-Based Hints

The list is the source of truth. A key is in the data structure only if a live node with that key can be reached by following `next` pointers from `head`. The BST does not decide whether a key is present. It only stores pointers to list nodes so that operations can start their search near the target key.

Because the BST is only a hint, it does not have to be perfectly up to date. It may return a stale node, miss a new node, or return a node that is not very close to the target. This is still safe because the operation does not trust the BST hint as the final answer. It uses the hint only as a starting point, then checks the relevant part of the list before it returns or performs an update.

This separation makes the design easier to reason about. All linearization points are in the list. The tree only affects how fast an operation reaches the right part of the list. This is also why the unbalanced and balanced BST hints can be swapped without changing the list algorithm.

3.2 Node representation

Each list node contains a key, a forward pointer, and a backward pointer. The forward pointer stores both a node pointer and a small state tag. The tag is stored in unused low pointer bits, which is possible because nodes are aligned.

Table 1: State tags stored in a list `next` pointer.

State	Value	Meaning
`Ok`	0	The node is live.
`Deleted`	1	The node has been logically removed.
`ExtMin`	2	The node is claimed by an `extract_min`.
`ExtMax`	3	The node is claimed by an `extract_max`.

```
enum State : uint8_t { Ok = 0, Deleted = 1, ExtMin = 2, ExtMax = 3 };

struct MarkedPtr {
    Node* ptr;
    State state;
};

struct alignas(64) Node {
    std::atomic<uintptr_t> prev;  // backward hint
    std::atomic<uintptr_t> next;  // pointer plus State tag
    long key;
};
```

Listing 1: Simplified list-node layout.

The forward pointer is correctness-critical. Successful compare-and-swap operations on `next` are the linearization points of updates and destructive extrema operations.

The backward pointer is only a hint, and the two list variants store it differently. The barrier-based variant stores a pointer to a `PrevWrapper`, which lets it use compare-and-swap

repairs while avoiding ABA ambiguity. The store-based variant stores a raw `Node*` directly and overwrites it with a plain atomic store. Both representations fit in one pointer-sized word. The two repair schemes are compared in the backward-pointer maintenance section.

3.3 The four variants

The implementation combines two list backends with two hint indexes. All four variants allow for the same operations.

Table 2: Implemented variants.

Variant	List backend	Hint index
`BSTList`	Barrier-based hinted list	Unbalanced BST
`BSTListBalanced`	Barrier-based hinted list	Balanced chromatic BST
`BSTListPragmatic`	Store-based hinted list	Unbalanced BST
`BSTListBalancedPragmatic`	Store-based hinted list	Balanced chromatic BST

Changing the hint backend does not change the list algorithms. Changing the list backend only changes backward-pointer maintenance. The barrier variant uses compare-and-swap repairs over wrapper objects. The store-based variant uses a single plain atomic store of a bare `Node*`.

4 Operations

4.1 Search, insert, and remove

A list search returns adjacent nodes `pred` and `curr` such that `pred.key < key <= curr.key`. The search may start from a hint node returned by the BST. If the hint is stale, deleted, or not useful, the search falls back to a safe traversal from `head`. During traversal, deleted nodes may be unlinked so that future traversals are shorter.

Insertion follows the usual lock-free list pattern. A new node is initialized with `prev = pred` and `next = curr`. The operation then tries to splice it into the forward chain with a compare-and-swap on `pred->next`. This compare-and-swap is the linearization point. Removal first locates the node and then changes its state from `Ok` to `Deleted`. The successful state-changing compare-and-swap is the linearization point. Physical unlinking can be done later by the same thread or by a helper.

Algorithm 1 High-level insertion.

```
1: procedure INSERT(key)
2:     hint ← FINDCLOSE(key)
3:     loop
4:         (pred, curr) ← LISTFIND(hint, key)
5:         if curr.key = key then
6:             return false
7:         end if
8:         n ← a new node with n.key = key, n.prev = pred, n.next = curr
9:         if CAS(pred.next, curr, n) succeeds then
10:            repair curr.prev as a hint
11:            update the BST hint index best-effort
12:            return true
13:        end if
14:    end loop
15: end procedure
```

4.2 Minimum and maximum queries

The `min` query scans from `head` until it finds the first live node. Since the list is sorted, this node is the minimum.

The `max` query uses `tail->prev` as a starting hint and walks backward through `prev` pointers. At each node, it reads the node's `next` state. If the state is `Ok`, the node is live, and because the walk started from the tail side of the sorted list, this node is returned as the maximum. If the state is `Deleted`, `ExtMin`, or `ExtMax`, the node has already been claimed or removed, so the query continues backward.

This backward walk affects memory management. A `prev` pointer is only a hint, so it can point to a node that has already been removed from the forward list. If such a node were freed while the program is still running, a later `max` or `extract_max` could follow the stale hint and access freed memory.

For this reason, removed list nodes are not freed during a benchmark run. A remove operation first marks a node as `Deleted`. Later, when a thread encounters that deleted node while traversing the list, it may physically unlink it from the forward chain. However, even after physical unlinking, the node's memory is kept until the data structure is destroyed. Section 6.3 explains this choice in more detail.

Algorithm 2 High-level `max`.

```
1: procedure MAX
2:     cur ← tail->prev
3:     while cur ≠ head and cur ≠ null do
4:         if cur.next.state is Ok then
5:             return cur.key
6:         end if
7:         cur ← cur.prev
8:     end while
9:     return NOKEY
10: end procedure
```

4.3 Extracting the minimum

The `extract_min` operation removes and returns the smallest live key in the list. The main difficulty is that several threads may call `extract_min` at the same time. The implementation must make sure that two threads do not both remove and return the same minimum node.

To do this, `extract_min` uses a short epoch stored in `head->next`. A thread opens the epoch by changing the state of `head->next` from `Ok` to `ExtMin`. While this epoch is open, other `extract_min` operations do not choose their own minimum. Instead, they help finish the open extraction.

After opening the epoch, the operation scans forward from `head`. It skips nodes that are already logically deleted and looks for the first live node. This node is the current minimum because the list is sorted.

The operation then claims this node by changing the state of its `next` pointer from `Ok` to `ExtMin`. This successful stamp is the linearization point of the extraction. After the node has been stamped, no other operation can successfully remove and return the same node. The operation then advances `head->next` past this node, closes the epoch, and returns the node's key.

If the scan reaches `tail`, then there is no live node in the list. In that case, the operation closes the epoch and returns NoKey.

Algorithm 3 High-level `extract_min`.

```
 1: procedure ExtractMin
 2:     loop
 3:         h ← read head->next
 4:         if h.state is ExtMin then
 5:             HelpExtractMin
 6:             continue
 7:         end if
 8:         if opening the epoch with CAS fails then
 9:             continue
10:         end if
11:         v ← first live node after head
12:         if v is tail then
13:             close the epoch
14:             return NoKey
15:         end if
16:         if stamping v.next with ExtMin succeeds then
17:             unlink v and close the epoch
18:             return v.key
19:         end if
20:         help close the epoch and retry
21:     end loop
22: end procedure
```

4.4 Extracting the maximum

The `extract_max` operation does not use a global epoch. Each call searches for the rightmost live node and tries to stamp that node's next pointer with `ExtMax`. The stamp succeeds only if the node is still live and its successor is `tail`. The same compare-and-swap verifies that the node is the current maximum and claims it for the winning thread.

Only the thread that wins the `ExtMax` stamp may return the key. Other threads may help unlink an `ExtMax`-stamped node, but helping does not give them ownership of the abstract removal. This prevents duplicate returns.

The backward pointer `tail->prev` is only a starting hint. After a candidate maximum is found, the operation validates it on the forward chain. If the candidate is no longer the maximum, the operation retries.

5 Correctness Argument

The correctness argument relies on three invariants.

1. The forward chain from `head` to `tail` is the real representation of the list.
2. A live key is represented by exactly one node whose next-pointer state is `Ok`.
3. Backward pointers and BST entries are hints. They can affect traversal time, but they do not decide membership. Updates and destructive extrema operations linearize through checks on the forward `next` pointers.

5.1 Linearization points

Table 3: Linearization points.

Operation	Linearization point
`insert(k)`	Successful CAS that links the new node after its predecessor.
`remove(k)`	Successful CAS that changes the node state from `Ok` to `Deleted`.
`min()`	Read that identifies the first live node after `head`.
`max()`	Read of the first live node found by the backward walk from `tail`.
`extract_min()`	Successful CAS that stamps the chosen node with `ExtMin`.
`extract_max()`	Successful CAS that stamps the chosen node with `ExtMax`.

For `insert`, the linearizing compare-and-swap checks that the predecessor still points to the successor found during the search. Since the search found the sorted position on the forward chain, the new node appears atomically between those two nodes.

For `remove`, the linearizing compare-and-swap changes the target node from live to logically removed. Later physical unlinking does not affect the abstract list because the key is already absent after the logical deletion mark is installed.

For `extract_min`, the head epoch ensures that only one minimum extraction chooses the first live node at a time. Stamping this node with `ExtMin` is the abstract removal. Helpers may complete the unlinking, but only the operation that owns the epoch returns the stamped key.

For `extract_max`, the `ExtMax` stamp checks that the chosen node's successor is `tail`. At the successful stamp, this node is the maximum live key.

5.2 Lock-freedom

All progress-critical changes are compare-and-swap operations on forward `next` pointers or their state tags. A failed compare-and-swap means that another thread changed the relevant word, which is system-wide progress. Backward-pointer repairs are not waited on. The barrier variant may fail and retry through the outer operation. The store-based variant uses a single plain atomic store that cannot fail. Helping in `extract_min` and during physical removal prevents a delayed thread from blocking the rest of the system.

6 Implementation Notes

6.1 Backward-pointer maintenance

The two list backends differ only in how they update the backward `prev` pointer. The forward chain is the same in both variants.

The barrier-based backend repairs `prev` with compare-and-swap. Since compare-and-swap checks the old value before writing the new one, it can be affected by the ABA problem: a pointer may change away from a value and later change back to the same value. To make such changes visible, the barrier-based backend stores `PrevWrapper*` values instead of raw `Node*` values. Each repair allocates a fresh wrapper, so even the same logical predecessor has a different wrapper address after a later write. The old wrapper is then retired through DEBRA.

The store-based backend follows the pragmatic approach of Träff and Pöter [7]. It does not use compare-and-swap on `prev`. Instead, it writes the new predecessor with one atomic store of a raw `Node*`. Since there is no compare-and-swap on `prev`, there is no ABA check to protect. This removes the wrapper allocation and reclamation cost from backward-pointer repair.

```
// Barrier variant: CAS in a fresh wrapper,
// then retire the displaced wrapper through DEBRA.
bool cas_prev(Node* x, uintptr_t expected, Node* z) {
    PrevWrapper* wrapper = new PrevWrapper(z);
    uintptr_t desired = reinterpret_cast<uintptr_t>(wrapper);
    if (!x->prev.compare_exchange_strong(expected, desired)) {
        delete wrapper;
        return false;
    }
    retirePW(reinterpret_cast<PrevWrapper*>(expected));
    return true;
}

// Store-based variant: one atomic store of a raw Node*.
// No CAS, no wrapper allocation, and no wrapper reclamation.
void store_prev_direct(Node* x, Node* z) {
    x->prev.store(reinterpret_cast<uintptr_t>(z),
                  std::memory_order_release);
}
```

Listing 2: Barrier-based and store-based backward-pointer repair.

Both variants are correct because `prev` is never used to prove membership or adjacency. A stale `prev` pointer can make an operation start farther from its target, but the operation still validates the final position using the forward chain. The store-based backend uses this fact directly: since `prev` is only a hint, a plain store is enough.

6.2 Balanced BST hint implementation

The balanced hint index stores keys in the leaves of the chromatic tree. Each leaf contains a pointer to the corresponding list node. Internal nodes are used only to guide the search. A few sentinel nodes keep the top of the tree stable, as in the SetBench chromatic-tree implementation.

The wrapper exposes the same interface as the unbalanced hint index. A lookup returns a nearby list node, and the list operation then continues from that node. Insertions and removals update the hint index on a best-effort basis. If the tree returns an old pointer or misses a recent update, the list traversal still determines the correct result.

The CBAT augmentation mechanism is not used. In particular, this implementation does not maintain version trees, subtree sizes, rank information, or range-query summaries. The balanced tree is included only to test whether a better-shaped hint index improves the BST+List design.

6.3 Memory reclamation

The memory reclamation code is based on the SetBench implementation of DEBRA, an epoch-based reclamation scheme [1, 8]. In an epoch-based scheme, each operation enters a critical section before reading shared nodes and leaves it after it no longer holds data-structure pointers. A retired object is freed only after all operations that could have seen it have left their critical sections.

DEBRA works well for nodes reached through the live structure. For example, a thread that follows the forward `next` chain from `head` inside a critical section is protected from seeing freed nodes. The problem in this project is that list nodes can also be reached through stale hints. A `prev` pointer or a list-node pointer stored in the BST hint index may still point to a node that has already been physically removed from the forward list. These pointers are only hints, so they are not repaired immediately when a node is removed. A later `max`, `extract_max`, or hint-based search may still read one of them. If the removed list node had already been freed, this read could become a use-after-free. This happened under AddressSanitizer in both list variants.

For this reason, the implementation does not free list nodes during a run. A removed node is still logically deleted and physically unlinked from the forward chain, so the live list shrinks correctly. The only delayed step is the memory free. Removed list nodes are stored in per-thread deferred sets and are freed when the data structure is destroyed. This makes every stale backward pointer and every stale BST hint point to allocated memory. The cost is memory usage, not operation time.

`PrevWrapper` objects exist only in the barrier-based backend. Each successful barrier repair installs a new wrapper and retires the old one through DEBRA. This is safe because a wrapper is read as the current value of a live node's `prev` field inside a critical section. Once a newer repair replaces it, DEBRA delays freeing the old wrapper until all threads that could have read it have left their critical sections. The store-based backend has no wrapper objects. Its `prev` field stores a raw `Node*`, so there is no extra object to retire or reclaim for backward-pointer repairs.

Tree nodes in both hint indexes can be reclaimed during a run. A lookup starts from the tree root and follows child pointers inside a DEBRA critical section, so DEBRA protects the tree nodes that the lookup reaches. Once a tree node is unlinked from the tree, a new lookup starting from the root cannot reach it again. Only operations that already held a pointer to that node need to finish, and DEBRA's epoch delay handles that case. When a delete removes tree nodes from the live tree, those nodes are retired through DEBRA and freed after no active operation can still hold them.

The balanced hint index still keeps its SCX descriptor records until teardown. These records can remain reachable by helpers after the update that created them. Reclaiming them during the run would require the descriptor-reclamation machinery used by the SetBench record manager. The tree nodes themselves are still reclaimed during the run.

One small issue remains in teardown. If `extract_min` skips over already deleted nodes near the head, those skipped nodes can become unreachable from the head without being registered for later freeing. AddressSanitizer reports this as a small bounded leak in the test suite. It is not a use-after-free and it does not affect the live structure. Section 9 describes a cleanup strategy.

7 Evaluation

7.1 Experimental setup

Benchmarks were run on the EPFL `lpdquad` server with 96 logical cores, four Intel Xeon E7-4830 v3 processors at 2.10 GHz, 504 GiB RAM, Linux 6.8.0, and `g++ -std=c++17 -O2 -pthread`. Each timed run lasts three seconds. The data structure is pre-filled with approximately half of the key range before measurement begins. Throughput is reported in million operations per second (Mops/s).

The main mixed workload uses 50% inserts, 20% removes, 10% `min`+`max` queries, 10% `extract_min`, and 10% `extract_max`. Inserts are more frequent than removals and extracts combined, so the structure usually remains non-empty during the run. Additional benchmarks isolate insert, remove, `min`+`max`, `extract_min`, and `extract_max`. Operation attempts are counted even when an insert finds a duplicate key, a remove misses, or an extract sees an empty structure.

The CBAT comparison in section 7.4 was produced by a separate benchmark binary compiled with `-O3` for all variants in that run.

7.2 Baseline selection

A direct baseline would be an existing non-blocking data structure that supports the same six operations: `insert`, `remove`, `min`, `max`, `extract_min`, and `extract_max`. We did not find an existing implementation with exactly this interface. Harris's list [4] and the Natarajan-Mittal BST [6] support updates and membership queries, but not efficient extrema operations. CBAT [9] supports augmented queries such as `min` and `max`, but it does not provide atomic destructive operations such as `extract_min` and `extract_max`.

CBAT is still a useful comparison for the non-destructive part of the workload. Its augmentation mechanism can maintain extrema information in the tree. For example, if each subtree stores its minimum and maximum key, `min` and `max` can be answered from the root in $O(1)$ time, as in the list-based design. This makes CBAT relevant for workloads containing `insert`, `remove`, `min`, and `max`, even though it is not a full baseline for destructive extrema operations.

7.3 Mixed workload

Table 4 summarizes the mixed-workload results, now including `extract_min` and `extract_max`. The unbalanced hint index consistently outperforms the balanced hint index. At key ranges 1,000 and 10,000, the store-based variant leads at every thread count. At key range 1,000, throughput peaks near 16 threads and falls at 32 because the single shared extraction state in `head->next` becomes a contention bottleneck. At key range 100,000 the two unbalanced variants are within a few percent of each other and trade a narrow lead, because forward list traversal, not back-pointer repair, dominates the running time. The balanced variants remain slower at all key ranges.

Table 4: Mixed throughput in Mops/s.

Key range	Threads	`BSTList`	`Balanced`	`Pragmatic`	`BalancedPrag`
1,000	1	0.823	0.351	**0.921**	0.386
1,000	2	1.416	0.510	**1.623**	0.543
1,000	4	1.719	0.617	**2.037**	0.422
1,000	8	2.828	0.365	**3.082**	0.576
1,000	16	4.936	0.930	**5.572**	0.611
1,000	32	4.065	0.473	**4.532**	0.465
10,000	1	0.073	0.035	**0.076**	0.036
10,000	2	0.139	0.065	**0.142**	0.066
10,000	4	0.262	0.112	**0.269**	0.114
10,000	8	0.468	0.200	**0.486**	0.206
10,000	16	0.837	0.355	**0.881**	0.362
10,000	32	1.357	0.468	**1.378**	0.459
100,000	1	**0.004**	0.002	0.003	0.002
100,000	2	**0.007**	0.005	**0.007**	0.005
100,000	4	**0.015**	0.009	**0.015**	0.009
100,000	8	**0.028**	0.017	**0.028**	0.017
100,000	16	**0.054**	0.032	**0.054**	0.032
100,000	32	0.097	0.057	**0.098**	0.055

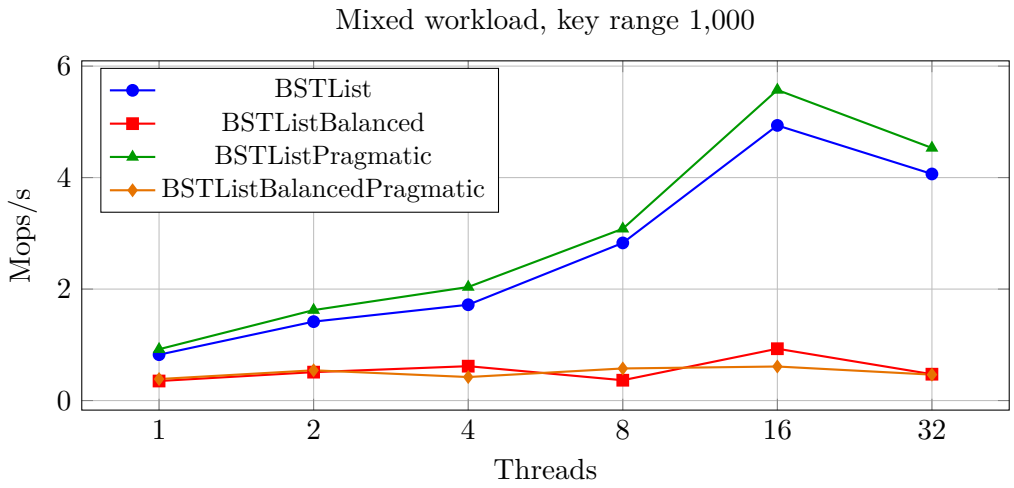


Figure 1: Mixed throughput at key range 1,000 (workload includes `extract_min` and `extract_max`). The store-based variant leads at every thread count. Both balanced variants are slower, and throughput peaks at 16 threads before falling at 32 because `extract_min` contention dominates.

The main result is that balanced hints are not useful in these workloads. They pay LLX/SCX synchronization and rebalancing cost on every update, but the operation still has to validate its result in the list. The extra tree cost is larger than the traversal time saved by the better hint.

The difference between the two unbalanced variants is smaller than the difference between balanced and unbalanced hints. At key ranges 1,000 and 10,000, the store-based variant leads at every thread count. Its backward-pointer repair is a single atomic store of a raw `Node*`. The barrier-based variant must allocate a wrapper, install it with compare-and-swap, and retire the old wrapper. This extra work is most visible when the list is short enough that traversal is not yet the dominant cost.

At key range 100,000, the two unbalanced variants are almost the same. This is because the main cost becomes forward traversal from the hint to the target node. That traversal is shared by both variants and grows with the live-set size. The cost that differs between the variants is only the backward-pointer repair, which is constant per update. As the list becomes longer, this constant difference becomes a small part of the total operation time.

7.4 Comparison with a lock-free balanced tree

Table 5 compares `BSTListPragmatic` with CBAT [9]. CBAT is a lock-free balanced chromatic tree with augmentation support. It supports `insert`, `remove`, `min`, and `max`, but not `extract_min` or `extract_max`. Therefore, this comparison uses a workload with 40% `insert`, 40% `remove`, and 20% `min`+`max`. Destructive extrema operations are excluded because CBAT does not support them.

Table 5: Mixed throughput in Mops/s: store-based BST+List vs. CBAT.

	Key range 1,000		Key range 10,000	
Threads	`BSTListPragmatic`	`CBAT`	`BSTListPragmatic`	`CBAT`
1	**0.790**	0.184	0.040	**0.129**
2	**1.501**	0.275	0.068	**0.197**
4	**2.039**	0.395	0.117	**0.237**
8	**3.564**	0.600	0.206	**0.409**
16	**7.137**	1.174	0.348	**0.808**

At key range 1,000, `BSTListPragmatic` is faster than CBAT at every thread count. The live list has about 500 keys, so the list walk after the hint lookup is short and cache-friendly. CBAT still pays for descriptor allocation, LLX/SCX synchronization, and chromatic-tree maintenance.

At key range 10,000, CBAT becomes faster. The reason is the cost of updates. An update in the BST+List design has two stages: first a tree lookup for a hint, then a list traversal from that hint to the exact position. CBAT performs one balanced-tree traversal and updates the tree directly. As the key range grows, the list traversal becomes the main cost.

The `min` and `max` operations are not the bottleneck in this comparison. CBAT can answer them in $O(1)$ by storing extrema in the augmentation at the root. The BST+List design also answers them from the sentinels and backward hints. The performance difference comes mainly from `insert` and `remove`.

This comparison should be read carefully. CBAT is a strong baseline for `insert`, `remove`, `min`, and `max`. It is not a full baseline for this project because it does not support `extract_min` or `extract_max`, as discussed in section 7.2.

7.5 Per-operation results

Table 6 isolates the main operation types at 8 threads and key range 10,000. The unbalanced variants are much better for destructive operations. `BSTListPragmatic` leads `extract_min` and `extract_max`. The balanced variants are slower because the balanced hint index adds LLX/SCX and rebalancing overhead.

The `min`+`max` throughput is much higher than update throughput because these queries are short and read-only. They mainly read the ends of the list instead of traversing to a random key or modifying the structure. These measurements are also sensitive to where the benchmark threads run on the machine. The server has several CPU sockets, and moving threads across sockets can change cache and memory-access costs. To reduce this noise, the threads for this benchmark were fixed to cores on one socket. Under this setup, all four variants are close.

Table 6: Per-operation throughput in Mops/s, 8 threads, key range 10,000.

Variant	insert	remove	min+max	extract_min	extract_max
`BSTList`	0.063	0.061	27.345	1.402	1.827
`BSTListBalanced`	0.067	0.042	30.451	0.249	0.237
`BSTListPragmatic`	**0.075**	**0.066**	30.423	**1.917**	**2.250**
`BSTListBalancedPragmatic`	0.068	0.043	30.394	0.293	0.263

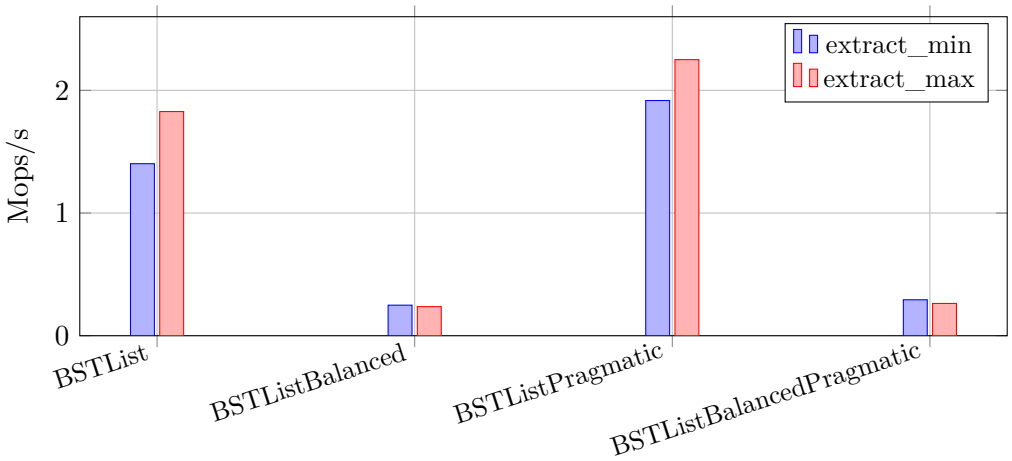


Figure 2: Extract throughput at 8 threads and key range 10,000.

8 Discussion

The experiments suggest three conclusions.

First, the store-based list is usually the stronger list backend for destructive extrema operations. It avoids compare-and-swap loops on backward-pointer repair and therefore reduces synchronization overhead. This is visible in the per-operation results for `extract_min` and `extract_max`.

Second, the balanced hint index does not help in these workloads. A chromatic tree gives better worst-case height than an unbalanced BST, but it also adds LLX/SCX synchronization and rebalancing. Since the tree is only a hint, the operation still has to validate its result in the list. In the measured workloads, the extra tree cost is larger than the traversal time saved by the better hint.

Third, the design works best when the list is not too large or when the workload contains many extrema operations. At larger key ranges, `insert` and `remove` spend more time traversing the list after receiving a hint. This is the main tradeoff of keeping correctness in the list and using the tree only as a hint structure.

The store-based backend leads the barrier-based backend in the mixed workload at the two smaller key ranges. The reason is that its backward-pointer repair is cheaper: it uses one atomic store of a raw `Node*`, while the barrier-based backend uses compare-and-swap over wrapper objects. At the largest key range, this advantage mostly disappears because both variants spend most of their time traversing the forward list.

9 Limitations and Future Work

The current implementation has several limitations. The hint index is only approximate, and the measured hint quality is not strong enough to remove long list traversals at large key ranges. The balanced hint index also has high overhead because it performs full chromatic-tree rebalancing even though the list, not the tree, defines correctness. Finally, list nodes and the balanced hint index's SCX descriptor records are freed only at teardown, which increases memory usage in long runs.

A cheaper balanced hint structure. A full lock-free chromatic tree may be more powerful than needed for a hint index. A future version could use a cheaper approximate index, such as a relaxed skip-list hint, a sampled index, or a periodically rebuilt tree. Since the index is only a hint, it may not need to rebalance after every update.

A single allocation list for list nodes. List nodes are freed only at teardown. A cleaner design would register every published list node in a per-thread allocation list and free those lists after all threads stop. This would also fix the small teardown leak caused when `extract_min` skips deleted nodes near the head.

Runtime reclamation of SCX descriptor records. Tree nodes in both hint indexes are reclaimed during a run (section 6.3). The remaining deferred objects are the SCX descriptor records used by the balanced hint index. These records can stay reachable after the update that created them, so freeing them during the run would need additional descriptor-reclamation support. A future implementation could adapt the SetBench record manager used by CBAT for this purpose. This is less urgent than list-node reclamation because it only affects the balanced hint index.

Adaptive back-pointer repair. The current implementation chooses one repair strategy globally. A future version could adapt at runtime. It could use store-based repair in low-contention cases and switch to compare-and-swap repair when backward links become too stale. It could also repair a small fixed number of backward pointers near `tail` when `max` or `extract_max` repeatedly finds stale hints.

10 Conclusion

This project implemented and evaluated a lock-free ordered list with BST hints. The central design choice is to keep correctness in the list and use the tree only to find good starting points for traversal. This makes stale tree entries safe and allows different hint indexes to be used without changing the list-level correctness argument.

The store-based unbalanced variant is strongest for destructive extrema operations and is the better list backend in the mixed workload at the two smaller key ranges. At the largest key range, the two unbalanced variants converge because forward traversal dominates the running time. The balanced hint index does not improve throughput in the tested configurations, which suggests that a hint structure should be much cheaper than a full balanced lock-free tree. The comparison with CBAT shows the main tradeoff: a true balanced tree is better for large update-heavy workloads, while the BST+List design provides efficient destructive extrema operations that CBAT does not support.

Declaration of AI Use

Claude Code was used as an AI coding assistant during the project. It helped generate code skeletons, translate pseudocode into C++, identify possible bugs, and draft preliminary text. The algorithmic choices, correctness reasoning, experiments, and final report were reviewed and edited by the author.

References

[1] Trevor Brown. Reclaiming memory for lock-free data structures: There has to be a better way. In *Proceedings of the 2015 ACM Symposium on Principles of Distributed Computing*, pages 261–270, 2015.

[2] Trevor Brown, Faith Ellen, and Eric Ruppert. A general technique for non-blocking trees. In *Proceedings of the 19th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 329–342, 2014.

[3] Mikhail Fomitchev and Eric Ruppert. Lock-free linked lists and skip lists. In *Proceedings of the 23rd Annual ACM Symposium on Principles of Distributed Computing*, pages 50–59, 2004.

[4] Timothy L. Harris. A pragmatic implementation of non-blocking linked-lists. In *Proceedings of the 15th International Symposium on Distributed Computing*, pages 300–314, 2001.

[5] Vladimir Melnikov and Gal Sela. Concurrent extrema and successor/predecessor queries in search trees. Semester project report, EPFL/Technion, 2026.

[6] Aravind Natarajan and Neeraj Mittal. Fast concurrent lock-free binary search trees. In *Proceedings of the 19th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 317–328, 2014.

[7] Jesper Larsson Träff and Martin Pöter. A more pragmatic implementation of the lock-free, ordered, linked list. arXiv:2010.15755, 2020.

[8] Trevor Brown. SetBench: A benchmark suite and code library for concurrent sets and maps. `https://gitlab.com/trbot86/setbench`, 2020.

[9] Evan Wrench, Ajay Singh, Younghun Roh, Panagiota Fatourou, Siddhartha Jayanti, Eric Ruppert, and Yuanhao Wei. Concurrent balanced augmented trees. In *Proceedings of the 31st ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, 2026.